

Football Stats Web

QA / Test Guide

Обновлено: 14.04.2026

Подробное описание бизнес-логики веб-части для тестировщика

Это tester-facing документ по текущему состоянию фронтенда проекта `football-stats-web`. Ниже разобраны реальные сценарии работы страниц, роль каждого компонента, его API-вызовы, реакция интерфейса на успешный и ошибочный ответ, а также важные ограничения и legacy-участки, которые не должны приниматься за основную боевую логику.

Основной стек

Vue 3 + Vite + Vue Router

Главный принцип

Страница = сценарий + API + роль

Важный акцент

Часть экранов legacy и не равна production-flow

Содержание

1. [Что это за продукт и зачем он нужен](#)
2. [Область тестирования и карта данных](#)
3. [Сквозная бизнес-логика приложения](#)
4. [App shell, авторизация и маршрутизация](#)
5. [Страница туров и таблицы сезона](#)
6. [Страница матча и протокол](#)
7. [Страница игроков и модальное окно карточки](#)
8. [Страница команд](#)
9. [Админ-панель](#)
10. [Кабинет представителя команды](#)
11. [API Explorer и экран теста endpoint](#)
12. [Legacy и неосновные части фронтенда](#)
13. [Сводная матрица страниц, API и триггеров](#)
14. [Замечания для QA](#)

Вводный бизнес-контекст проекта

Football Stats Web - это веб-интерфейс для локального футбольного турнира, который объединяет публичную витрину соревнования, операционную работу организаторов и рабочее место представителей команд. По сути, система нужна для того, чтобы в одном месте вести сезон, публиковать туры, собирать составы команд на матчи, оформлять официальные протоколы и показывать участникам и болельщикам актуальную турнирную картину.

Для кого продукт

Для организаторов турнира, представителей команд, игроков, зрителей и всех, кому нужен прозрачный статус сезона и матчей.

Что решает

Убирает разрозненный учет в чатах и таблицах, фиксирует официальный состав матча, результат и историю сезона в едином интерфейсе.

Главная сущность

Сезон турнира: внутри него есть команды, автоматически сформированные туры, матчи, заявки, протоколы и турнирная таблица.

Ожидаемый результат

После подтверждения протокола матч становится официально завершенным, а публичная таблица и статистика отражают факт турнира.

Какая бизнес-модель зашита в продукт

- Организатор создает сезон и задает его формат: количество кругов, наличие плей-офф, состав участников.
- На основе состава команд и настроек формата backend формирует сетку регулярных туров.
- Организатор назначает матчи по турам и публикует туры, после чего они становятся видны на публичной главной странице.
- Представитель команды ведет свой roster и формирует сезонную заявку, а затем подает заявку на конкретный матч.
- Администратор на основе уже поданных составов оформляет официальный протокол матча: счет, голы, карточки, техническое поражение при необходимости.
- После подтверждения протокола матч становится источником истины для standings, статистики игроков и общей картины сезона.

Кто и как взаимодействует с системой**Организатор / SUPER_ADMIN**

Управляет справочниками и турнирным контуром: сезоны, команды, туры, матчи, игроки, роли и окончательное утверждение результата матча.

Представитель команды / TEAM_REP

Поддерживает актуальный состав своей команды, переводит игроков в заявку сезона и подает заявку на матч от имени своей команды.

Публичный пользователь

Сматривает календарь, турнирную таблицу, карточки матчей и игроков. Для него система выступает как официальный информационный сайт турнира.

Если описать продукт одной фразой

Это цифровой контур управления футбольным сезоном: от настройки турнира и состава команд до официального протокола матча и публичного отображения результатов.

Почему этот вводный блок важен для тестировщика

Без него экраны выглядят как набор отдельных форм и таблиц. На практике же здесь есть единая предметная цепочка: сезон - тур - матч - заявка - протокол - таблица. Именно поэтому многие проверки нужно строить не по отдельному экрану, а по сквозному бизнес-сценарию.

1. Область тестирования и карта данных

Текущий фронтенд состоит из публичных страниц, role-based закрытых экранов и нескольких legacy-модулей. При тестировании важно различать, какие страницы живут на реальном backend API, а какие используют локальные мок-данные или вообще служат как

демонстрационные/вспомогательные экраны.

Реальный backend API

`/` , `/match/:id` , `/players` , `/admin` , `/team-rep-dashboard` , `/api-explorer` , `/api-explorer/test/:endpointKey` .

Смешанный режим

Экран API Explorer статичен по каталогу endpoint-ов, но тестовый экран выполняет реальные HTTP-запросы к backend.

Локальные данные

`/teams` , `/create` , `Home.vue` и часть `admin-ban` функционала работают на локальном `store/localStorage`.

Для QA-акцента

Тестировать как основной контур нужно публичные сезоны, матч, игроков, админку, кабинет TEAM_REP и API Explorer.

Главный принцип чтения этого документа

Сначала смотрим бизнес-смысл страницы, затем триггеры загрузки данных, затем матрицу endpoint-ов, затем граничные состояния. Такой порядок совпадает с тем, как тестировщик обычно строит чек-листы.

2. Сквозная бизнес-логика приложения

2.1 Роли и права

Гость / неавторизованный пользователь

Читает публичные страницы. Не видит защищенную навигацию. Может открыть API Explorer напрямую.

USER

Логически близок к гостю, но уже имеет JWT и может вызывать защищенные read-only endpoint-ы через токен.

TEAM_REP

Редактирует состав своей команды, управляет заявкой на сезон и подает заявку на матч за свою команду.

SUPER_ADMIN

Создает сезоны, команды, матчи, игроков, роли, team-score и заполняет/подтверждает протоколы матчей.

2.2 Общая модель работы с API

- Во фронтенде есть два основных wrapper-а: `authorizedApiRequest` и `optionalAuthApiRequest` .
- `authorizedApiRequest` падает сразу, если токена нет, и всегда добавляет `Authorization: Bearer ...` .
- `optionalAuthApiRequest` отправляет тот же запрос без токена, если пользователь не авторизован.
- Базовый слой `apiRequest` унифицирует обработку ответа: парсит JSON, выбрасывает `Error` по `response.ok === false` , маскирует чувствительные поля в dev-логах.
- Сетевые ошибки приводятся к единому сообщению: сервер недоступен.

2.3 Логи в dev-режиме

- `main.js` переопределяет глобальный `fetch` и логирует все API-запросы в консоль браузера.
- `auth.js` дополнительно логирует запросы, проходящие через wrapper-ы авторизации.

- В dev это может давать двойные логи по одному и тому же запросу. Для QA это не дефект бизнес-логики, а особенность инструмента диагностики.

2.4 Общая логика ошибок

- Каждая страница обычно держит локальный `errorText`, `pageError`, `messageError` или эквивалент.
- При неуспехе фронтенд не перезагружает весь SPA, а показывает текст ошибки внутри текущего экрана.
- После успешного действия почти везде есть повторная загрузка или локальная синхронизация состояния, чтобы UI не расходился с backend.

3. App shell, авторизация и маршрутизация

Файл `App.vue` определяет общий каркас сайта: верхнюю панель, навигацию, кнопку входа/регистрации, вывод имени пользователя либо ярлыка представителя команды, а также модальное окно авторизации.

Файл: `App.vue`

Маршрутизация: `router/index.js`

Auth store: `store/auth.js`

3.1 Бизнес-логика верхней панели

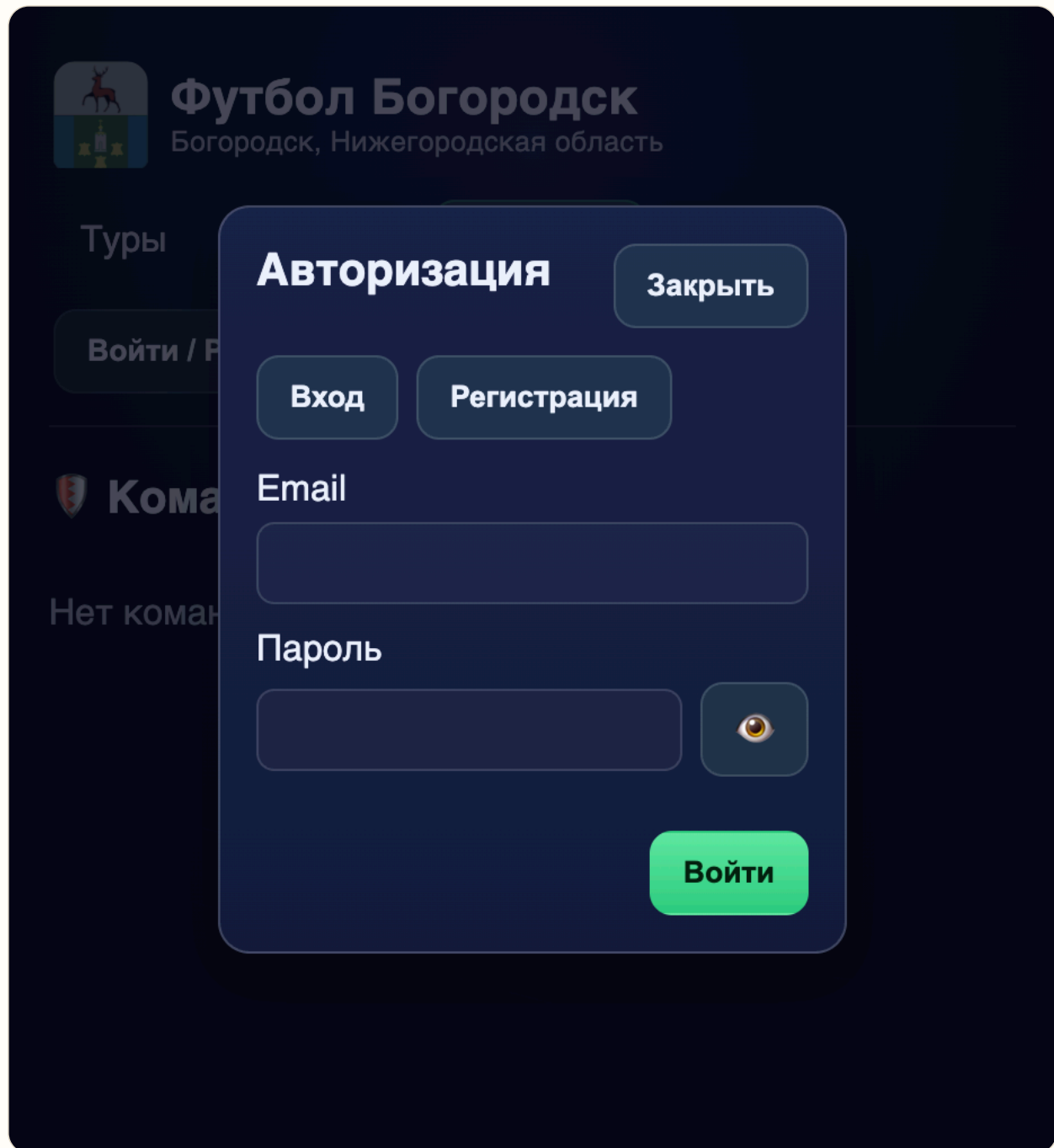
- Публично видны ссылки только на туры, игроков и команды.
- Ссылка на админ-панель и API Explorer в верхнем меню появляется только для `SUPER_ADMIN`.
- Для `TEAM_REP` вместо обычного имени показывается кнопка профиля вида `Представитель команды "..."`, ведущая в личный кабинет.
- При выходе из системы токен и пользователь удаляются из `sessionStorage`, после чего SPA принудительно уходит на главную страницу.

3.2 Модальное окно авторизации

ДЕЙСТВИЕ	ENDPOINT	ТРИГГЕР	РЕЗУЛЬТАТ UI
Регистрация	POST <code>/api/auth/register</code>	Вкладка «Регистрация», submit формы	Токен и профиль пишутся в <code>sessionStorage</code> , затем идет добор данных через <code>/api/auth/me</code> и <code>/api/admin/access/me</code> , модалка закрывается через 500 мс.
Вход	POST <code>/api/auth/login</code>	Вкладка «Вход», submit формы	Та же цепочка, что у регистрации, но без создания нового пользователя.
Добор текущего профиля	GET <code>/api/auth/me</code> GET <code>/api/admin/access/me</code>	После успешного auth и при загрузке приложения с уже сохраненной сессией	Фронтенд узнает роли, имя команды, <code>teamId</code> и флаги <code>canEditRoster</code> , <code>canEditApplication</code> .

3.3 Route guard

- `/create` требует просто авторизацию.
- `/admin` требует авторизацию и роль `SUPER_ADMIN`.
- `/team-rep-dashboard` требует авторизацию и роль `TEAM_REP`.
- `/api-explorer` фактически не защищен route guard-ом, хотя в верхнем меню показывается только администратору. Это важная особенность для QA.



Окно входа/регистрации. Это общая точка входа в аутентификацию для всего SPA. Отсюда нельзя выполнить гостевой логин: гостевой сценарий вынесен в API Explorer.

4. Страница туров и таблицы сезона

Маршрут `/` обслуживается файлом `Tours.vue` и является основной публичной стартовой страницей. Ее бизнес-задача: одним взглядом дать сезонный обзор, не заставляя пользователя отдельно ходить за таблицей, командами сезона и списком туров.

4.1 Что делает страница в бизнес-смысле

- Показывает список активных сезонов.
- После выбора сезона запрашивает aggregated overview объекта сезона.
- Отображает метаданные формата турнира: число кругов, число команд, число туров регулярки, наличие плей-офф.

- Слева рисует сохраненную турнирную таблицу сезона.
- Справа рисует только опубликованные туры с вложенными матчами.
- При раскрытии тура показывает матчи в порядке kickoff time.

4.2 API и триггеры

ENDPOINT	КОГДА ВЫЗЫВАЕТСЯ	ЧТО ИМЕННО ПРИХОДИТ	ЧТО ДЕЛАЕТ ФРОНТ
GET /api/seasons?active_flag=1	На onMounted	Список активных сезонов	Заполняет селект сезона. Если сезон еще не выбран, автоматически подставляет первый сезон в списке.
GET /api/seasons/{seasonId}/overview	При изменении выбранного сезона	Сезон, команды сезона, опубликованные туры, матчи внутри туров, standingsConfig , standings	Обновляет правую колонку с турами и левую таблицу standings без дополнительных запросов.

4.3 Встроенная бизнес-логика на фронте

- При смене сезона фронтенд сбрасывает раскрытый тур, чтобы не показывать несовместимое состояние от прошлого сезона.
- Туры сортируются по sortOrder , затем по roundNumber , затем по id .
- Матчи внутри тура сортируются по kickoffAt , затем по id .
- Для таблицы берутся уже готовые standings из backend. На клиенте таблица не пересчитывается.
- Статус матча отображается через локальный mapper: LIVE , FINISHED , VERIFIED , LINEUPS_SUBMITTED , default = запланирован.

Ключевой архитектурный смысл

Стартовая страница больше не пересчитывает таблицу на клиенте. Она только читает уже сохраненный snapshot сезона из backend. Для QA это означает, что проверять корректность standings нужно через цепочку: подтверждение протокола матча - пересчет в backend - новый overview.

Что обязательно проверить

Если таблица не совпадает с ожидаемым результатом после подтверждения протокола, проблема почти наверняка не в рендере Tours.vue , а в backend-пересчете или в содержимом /api/seasons/{id}/overview .

5. Страница матча и протокол

Маршрут /match/:id обслуживается файлом Match.vue . Это самый важный экран по бизнес-логике, потому что именно здесь сходятся публичный просмотр матча, работа представителя команды с заявкой и работа администратора с протоколом.

Публичный просмотр

TEAM_REP редактирует заявку

SUPER_ADMIN заполняет и подтверждает протокол

5.1 Бизнес-смысл экрана

- Показать базовую карточку матча: команды, логотипы, время, текущий счет, сезон, тур, статус протокола.
- Показать составы обеих команд и уже сохраненную статистику игроков в виде публичных иконок.
- Разрешить TEAM_REP подать либо скорректировать только свою заявку на матч.
- Разрешить SUPER_ADMIN перевести поданные составы в официальный протокол матча.

5.2 Основные запросы

ENDPOINT	КТО ВЫЗЫВАЕТ	КОГДА	РЕАКЦИЯ UI
GET /api/matches/{matchId}	Все роли	При первом открытии страницы и при локальном refresh	Заполняет весь экран: hero, lineups, available players, protocol, saved events.
PUT /api/matches/{matchId}/lineups/{teamId}	SUPER_ADMIN или TEAM_REP своей команды	Добавление игрока, удаление игрока, очистка заявки	Backend возвращает обновленный match payload, фронт сразу заменяет локальный match и синхронизирует draft протокола.
PUT /api/matches/{matchId}/protocol	SUPER_ADMIN	Сохранить или подтвердить протокол	Возвращается обновленный match с актуальным протоколом и событиями, далее фронт заново пересобирает draft.

5.3 Как работает логика заявки

- Для каждой команды рисуется отдельная карточка состава.
- Если у пользователя нет прав на эту команду, он видит только read-only список игроков и иконки статистики.
- Если права есть, доступны действия: добавить игрока, убрать игрока, очистить заявку.
- Перед открытием модальки добавления игрока фронтенд делает `refreshMatch()`, чтобы список доступных игроков был свежим и не расходился с backend.
- В payload заявки уходит не diff, а полный массив `playerIds` на команду.

5.4 Как работает draft протокола

- Фронтенд не хранит протокол в свободной форме. Он строит `protocolDraft` из текущего backend payload.
- Источник уже сохраненной статистики игрока - это не поля протокола как таковые, а список backend-событий `protocol.events`.
- Из этих событий собирается `savedStatsMap`, по которому и рисуются мячи и карточки возле игроков в публичной части.
- В админской части по тем же игрокам создаются редактируемые ячейки: голы, желтые, красные.

5.5 Ключевые валидации протокола

Нельзя сохранить протокол без двух заявок

Если обе команды не подали составы, кнопка сохранения выдаст ошибку про необходимость сначала подать обе заявки.

Сумма голов игроков должна совпасть со счетом

При обычном матче фронтенд сам считает голы по игрокам по каждой команде и сравнивает с полями счета.

Техническое поражение исключает индивидуальную статистику

При включении tech defeat все поля игрока обнуляются, а счет жестко становится 0:3 или 3:0.

5.6 Техническое поражение

- Выбор технического поражения одной стороны автоматически снимает тех.поражение у другой стороны.
- При включении флага блокируются ручные инпуты счета.
- При выключении обоих флагов счет сбрасывается в `0:0`, а редактирование ручного счета снова становится доступным.
- В публичной шапке матча поверх счета показывается отдельная подпись вида `Тех. пор. Команда`.

5.7 Что уходит в protocol API

- `status` = `FINISHED` при обычном сохранении и `VERIFIED` при подтверждении.
- `homeScore` и `awayScore` - либо вручную выставленные значения, либо принудительный `0:3 / 3:0` при tech defeat.

- `playerStats` - массив по каждому игроку состава, где фронт нормализует отрицательные значения в ноль.
- Поля `bestPlayerId`, `notes`, `startedAt`, `finishedAt` сейчас всегда отправляются как `null` и не участвуют в UI-сценарии.



Футбол Богородск

Богородск, Нижегородская область

[Турь](#)[Игроки](#)[Команды](#)[Войти / Регистрация](#)[← На главную](#)

Протокол подтвержден

Волга

17.04.2026, 23:09

1 : 2



Публичная часть карточки матча. В текущем состоянии без авторизации видно шапку матча, обе заявки и уже сохраненную статистику игроков. Админские элементы протокола на скриншоте отсутствуют, так как они появляются только для роли `SUPER_ADMIN`.

6. Страница игроков и модальное окно карточки

Маршрут `/players` обслуживается файлом `Players.vue`. Страница является публичным каталогом игроков с поиском, пагинацией и большим модальным окном детальной карточки игрока.

6.1 Бизнес-смысл

- Показать постраничный список игроков.
- Дать поиск по фамилии/имени без мгновенного спама в API.
- По клику на игрока открыть реальную карточку с фото, базовыми данными, суммарной статистикой и историей команд.

6.2 API-цепочка

ENDPOINT	КОГДА	НАЗНАЧЕНИЕ	РЕАКЦИЯ UI
<code>GET /api/players?pagenum=X&pagesize=20&name=QUERY</code>	При открытии страницы, при смене страницы, при изменении debounced поиска	Получить список игроков	Заполняет плоский список имен, пагинацию и счетчики.
<code>GET /api/players/{playerId}</code>	При открытии модалки игрока	Базовая карточка игрока	ФИО, фото, дата рождения, текущая команда, город, голы, ЖК, КК.
<code>GET /api/players/{playerId}/history</code>	При открытии модалки игрока	История команд	Параллельно наполняет нижний блок карточки.

6.3 Локальная логика

- Поиск завернут в `useDebounce(search, 1000)`, то есть API идет только через 1 секунду после остановки ввода.
- При изменении строки поиска номер страницы принудительно сбрасывается на 0.
- Карточка игрока грузит детали и историю параллельно через `Promise.all`.
- Если фото отсутствует, UI рисует крупный placeholder `Нет фото`, а не ломает layout.
- Если история пустая, вместо пустого блока показывается явный текст про отсутствие истории переходов.

Важная наблюдаемая особенность

В текущем runtime страница игроков работает публично даже без авторизации. При этом каталог API описывает часть player-endpoint-ов как auth-required. Для QA это значит, что реальное поведение и декларативное описание API надо сверять отдельно.



Футбол Богородск

Богородск, Нижегородская область

Турь

Игроки

Команды

Войти / Регистрация

Игроки

Сбросить поиск

Всего игроков: 58

Страница 1 из 3

Александр Тусов

Алексей Блохин

Андрей Крылов

Антон Белов

Антон Панкратов

Арсений Попов

Иван Иванов

Текущая команда: Центр

×

Артур Панов

Виктор Голубев

Иван Иванов

Владислав Рябов

Глеб Орлов

Григорий Мартынов

Даниил Брагин

Евгений Лавров

Егор Дмитриев

ФИО

Егор Климов

Иван Иванов

Егор Панов

ДАТА РОЖДЕНИЯ

Егор Шувалов

10.01.2000

Иван Иванов

ТЕКУЩАЯ КОМАНДА

Назад

Центр

Вперед

Живой пример карточки игрока: список, пагинация и уже открытая модалка с фото, статистикой и историей команды. Это не mock, а данные из `/api/players/{id}` и `/api/players/{id}/history`.

7. Страница команд

Маршрут `/teams` обслуживается файлом `Teams.vue`. На текущий момент это не полноценный backend-driven экран, а витрина, читающая `standings` из локального `store.js`.

7.1 Что это означает для тестировщика

- Страница не делает HTTP-запросов.
- Она не синхронизирована с новой season standings архитектурой, которая используется на главной странице.
- Если здесь пусто, это не означает, что в backend нет команд или турнирной таблицы.
- Этот экран нельзя использовать как эталон для проверки актуального season overview.



Футбол Богородск

Богородск, Нижегородская область

[Тур](#)[Игроки](#)[Команды](#)[Войти / Регистрация](#)

Команды

Нет команд в выбранном сезоне.

Текущее состояние экрана команд. Он может показывать пустое состояние независимо от того, что реально есть в backend, потому что живет на legacy store-данных.

8. Админ-панель

Маршрут `/admin` обслуживается файлом `Admin.vue` и открыт только для `SUPER_ADMIN`. Это главный back-office экран, разбитый на вкладки. Внутри каждой вкладки фронтенд реализует отдельный законченный business flow.

8.1 Состав вкладок

Сезоны

Создание, редактирование, деактивация сезона и полный набор команд сезона.

Команды

CRUD команд, логотипы, города и короткие названия.

Туры

Выбор сезона, список туров, добавление матчей, публикация тура, удаление матчей.

Игроки

Создание, редактирование и мягкое удаление игроков.

Роли

Поиск пользователя, выдача роли, замена роли, снятие роли через real admin-access API.

Представители

Поиск TEAM_REP, назначение команды, снятие привязки через team score.

Бан

Локальный frontend-only сценарий на localStorage, не backend-security механизм.

Сообщения

Все успехи и ошибки централизованно выводятся внизу панели через messageOk и messageError.

8.2 Основная логика по вкладкам

ВКЛАДКА	ОСНОВНЫЕ ENDPOINT-Ы	ЛОГИКА UI	ЧТО ВАЖНО ПРОВЕРИТЬ
Сезоны	POST /api/seasons PUT /api/seasons/{id} GET/PUT /api/seasons/{id}/teams DELETE /api/seasons/{id}	Есть отдельные подрежимы «создать» и «редактировать». Команды сезона формируются как явный список выбранных команд.	После сохранения должен обновляться сезонный реестр и при совпадении с выбранным сезоном тура - список туров.
Команды	GET /api/teams?active_flag=1 POST /api/teams PUT /api/teams/{id} DELETE /api/teams/{id}	Форма умеет читать изображение и конвертировать его в dataUrl до отправки.	Редактирование и деактивация должны обновлять список команд и косвенно влиять на выборы в сезонах/турах.
Туры	GET /api/tours?season_id=... GET /api/teams?season_id=... GET /api/tours/{tourId}/matches POST /api/tours/{tourId}/matches PUT /api/tours/{tourId}/publish DELETE /api/tours/{tourId}/matches/{matchId}	Сначала выбирается сезон, затем тур. До выбора тура экран матча не активен. Нельзя публиковать пустой тур.	На публичную главную попадают только опубликованные туры. Это критическая business rule.

ВКЛАДКА	ОСНОВНЫЕ ENDPOINT-Ы	ЛОГИКА UI	ЧТО ВАЖНО ПРОВЕРИТЬ
Игроки	<code>GET /api/players?active_flag=1&pagenum=0&pagesize=500</code> <code>POST /api/players</code> <code>PUT /api/players/{id}</code> <code>DELETE /api/players/{id}</code>	Есть create/edit режимы. Фото игрока также грузится как <code>dataUrl</code> .	После изменения игрока реестр должен сразу обновиться, а деактивация должна убирать игрока из списка редактирования.
Роли	<code>GET /api/admin/access/users</code> <code>GET /api/admin/access/users/{userId}</code> <code>POST/DELETE /api/admin/access/users/{userId}/roles/{roleCode}</code>	Поиск по email дебаунсится на 5 секунд, но есть и ручная кнопка «Найти». Замена роли делается последовательностью <code>revoke + assign</code> .	Проверять, что роль в UI реально обновляется после <code>refresh access</code> пользователя.
Представители	<code>GET /api/admin/access/users?role=TEAM_REP</code> <code>GET /api/admin/access/users/{userId}</code> <code>POST /api/admin/access/users/{userId}/team-scopes</code> <code>DELETE /api/admin/access/users/{userId}/team-scopes/{teamId}</code>	Перед назначением новой команды старые score-ы снимаются. UI явно предупреждает, если найдено несколько активных привязок.	После назначения представитель должен увидеть команду в своем кабинете и получить права на <code>roster/application</code> .
Бан	HTTP нет	Все хранится в <code>localStorage</code> под ключом реестра пользователей.	Не интерпретировать как реальный серверный механизм блокировки.

8.3 Нюансы, важные для QA

- При создании сезона фронтенд сначала создает сам сезон, затем отдельным запросом привязывает к нему набор команд.
- Регулярные туры не создаются вручную на фронте: UI лишь отображает рассчитанный backend-формат сезона.
- На вкладке ролей старые helper-функции `assignRole()` и `removeRole()` остаются в коде, но реальная вкладка использует server-side admin-access API.
- На вкладке бана ситуация обратная: там до сих пор используется локальная модель на `localStorage`, а не backend endpoint.

9. Кабинет представителя команды

Маршрут `/team-rep-dashboard` обслуживается файлом `TeamRepDashboard.vue` и закрыт route guard-ом по роли `TEAM_REP`. Это основной рабочий кабинет представителя команды, через который управляется текущий roster и seasonal application.

9.1 Бизнес-смысл кабинета

- Показать представителю его профиль и закрепленную команду.
- Показать список сезонов, в которых участвует его команда.
- Позволить редактировать текущий состав команды.
- Позволить по каждому сезону формировать отдельную заявку.
- Позволить создавать нового игрока и сразу добавлять его в состав команды.

9.2 Загрузка данных

ENDPOINT	КОГДА	ЧТО ДЕЛАЕТ
GET /api/team-rep/seasons	На старте кабинета и при ручном обновлении	Возвращает сезоны команды с агрегированными счетчиками по заявке.
GET /api/team-rep/players	На старте кабинета и после изменений состава	Возвращает текущий roster команды представителя.
GET /api/team-rep/seasons/{seasonId}/players	После выбора сезона, перед открытием модальки добавления игрока	Возвращает текущее представление seasonal application и список доступных игроков.

9.3 Управление заявкой на сезон

- После выбора сезона можно включить фильтр «Показать игроков в заявке», тогда в основном roster-блоке останутся только игроки, относящиеся к выбранному сезону.
- Кнопка «Добавить игрока» открывает модальку, где список собирается из двух источников: текущий roster, который еще не включен в сезон, и доступные внешние игроки без команды.
- Если выбран игрок уже из текущего состава, POST просто добавит его в сезон.
- Если выбран свободный игрок из `availablePlayers`, backend переведет его в команду представителя и одновременно добавит в заявку сезона.
- Удаление из сезона не удаляет карточку игрока и не отвязывает его от команды, а лишь снимает seasonal membership.

9.4 Управление текущим составом команды

- Удаление из команды идет через `DELETE /api/teams/{teamId}/players/{playerId}`.
- После удаления фронтенд делает `loadDashboard()`, а если выбран сезон - еще и `loadSeasonView()`, чтобы UI не оставил игрока в seasonal blocks.
- Создание и редактирование игрока выполняется через `/api/team-rep/players` и `/api/team-rep/players/{id}`.
- Фото игрока предварительно читается через `FileReader` и отправляется как `dataUrl`.

Ключевая бизнес-идея

В кабинете TEAM_REP различаются два уровня: текущий roster команды и заявка на конкретный сезон. Игрок может состоять в составе команды, но не входить в seasonal application. Именно это различие надо тестировать отдельно.

10. API Explorer и экран теста endpoint

Эти страницы обслуживаются файлами `ApiExplorer.vue` и `ApiExplorerTest.vue`. Они нужны одновременно как справочник по API и как встроенный ручной тестер запросов к backend.

10.1 API Explorer

- Строится не по live schema, а по статическому каталогу `src/data/apiExplorerCatalog.js`.
- Каталог является single source of truth для списка endpoint-ов, их описаний, параметров и примеров request body.
- Поэтому экран сам по себе не доказывает, что endpoint реально доступен. Он описывает ожидаемую контрактную модель.



Футбол Богородск

Богородск, Нижегородская область

Туры

Игроки

Команды

Войти / Регистрация

API индекс сервиса

http://127.0.0.1:8080

Ресурсы авторизации (4)

Метод	Путь	Параметры	Ог
POST	/api/auth/register	body	Сс во ро
POST	/api/auth/login	body	Во ак по
POST	/api/auth/guest	body	Вь бе чт
GET	/api/auth/me	—	Во ав по

Справочник команд (4)

Метод	Путь	Параметры
GET	/api/teams	active_flag, season_id
POST	/api/teams	body
PUT	/api/teams/{teamId}	teamId, body
DELETE	/api/teams/{teamId}	teamId

Сезоны (7)

Метод	Путь
GET	/api/seasons

POST`/api/seasons`**GET**`/api/seasons/{seasonId}/teams`**GET**`/api/seasons/{seasonId}/overview`**PUT**`/api/seasons/{seasonId}/teams`

PUT	/api/seasons/{seasonId}
DELETE	/api/seasons/{seasonId}
Туры и матчи тура (15)	
Метод	Путь
GET	/api/tours

GET	/api/tours
PUT	/api/tours/{tourId}/publish
GET	/api/tours/{tourId}/matches
POST	/api/tours/{tourId}/matches
GET	/api/matches/{matchId}

PUT`/api/matches/{matchId}/lineups/`**PUT**`/api/matches/{matchId}/protocol`

GET`/api/team-rep/seasons`**GET**`/api/team-rep/players`**POST**`/api/team-rep/players`**PUT**`/api/team-rep/players/{playerId}`

GET`/api/team-rep/seasons/{seasonId}`**PUT**`/api/team-rep/seasons/{seasonId}`**POST**`/api/team-rep/seasons/{seasonId}`

DELETE`/api/team-rep/seasons/{seasonId}`

Игроки (9)

Метод	Путь
GET	<code>/api/players</code>
GET	<code>/api/players/{id}/history</code>

GET	/api/players/{playerId}
POST	/api/players
PUT	/api/players/{playerId}
DELETE	/api/players/{playerId}
GET	/api/teams/{teamId}/players
POST	/api/teams/{teamId}/players/{pl

Индекс endpoint-ов. Здесь tester быстро видит группы ресурсов, HTTP-методы и может перейти на экран ручного выполнения запроса.

10.2 Экран теста конкретного endpoint

- Путь: `/api-explorer/test/:endpointKey`.
- На вход берется ключ endpoint-а и через `findEndpointByKey` поднимается metadata из каталога.
- Path params, query params и JSON body генерируются динамически исходя из метаданных.
- Перед отправкой фронт проверяет, что в URL не остались шаблоны `{param}`.
- Если endpoint помечен как `auth: true`, в запрос автоматически добавляется текущий JWT из auth store.
- Если JSON body невалиден, запрос вообще не отправляется, а пользователь получает ошибку в UI.



Футбол Богородск

Богородск, Нижегородская область

[Турь](#)[Игроки](#)[Команды](#)[Войти / Регистрация](#)

Гостевой вход

[Назад к списку](#)

Выдает JWT для гостя без регистрации. Права чтения как у USER.

Метод: **POST**

`/api/auth/guest`

Тело запроса (JSON)

[Сбросить](#)



Экран ручного прогона endpoint-а. На скриншоте показан сценарий гостевого входа. После нажатия «Тест» ниже появляется статус, время выполнения и тело ответа backend.

11. Legacy и неосновные части фронтенда

11.1 CreateMatch.vue

- Маршрут `/create` существует, но не входит в основную публичную навигацию.
- Сценарий работает через локальный `store.js`, а не через реальный backend.
- Форма принимает команды, счет, дату, лучшего игрока и строку бомбардиров, разбирая ее на клиенте.
- Этот экран не является текущим боевым способом создания матчей. Реальный production flow создания матча находится в админке на вкладке туров.

11.2 Home.vue

- Файл существует, но не используется роутером как текущая главная страница.
- Это legacy-экран старой модели: локальная таблица, последние матчи и топ-игроки на основе `store.js`.
- Проверить актуальное состояние турнира по нему нельзя.

11.3 store.js

- Хранит seeded demo-данные по турам, матчам и игрокам.
- Нужен для устаревших экранов и для части демонстрационной логики.
- Именно отсюда питаются страницы `/teams` и legacy flow `/create`.

12. Сводная матрица страниц, API и триггеров

СТРАНИЦА / КОМПОНЕНТ	РОЛЬ	ОСНОВНЫЕ API	ТРИГГЕР ВЫЗОВА	ОСНОВНОЙ БИЗНЕС-РЕЗУЛЬТАТ
App.vue auth modal	Все	<code>/api/auth/register</code> <code>/api/auth/login</code> <code>/api/auth/me</code> <code>/api/admin/access/me</code>	Submit формы входа/ регистрации, восстановление сессии	Авторизация, восстановление ролей и team-score пользователя.
Tours.vue	Публично	<code>/api/seasons?active_flag=1</code> <code>/api/seasons/{id}/overview</code>	Открытие страницы, смена сезона	Сезонный обзор: standings snapshot + опубликованные туры.

СТРАНИЦА / КОМПОНЕНТ	РОЛЬ	ОСНОВНЫЕ API	ТРИГГЕР ВЫЗОВА	ОСНОВНОЙ БИЗНЕС-РЕЗУЛЬТАТ
Match.vue	Публично / TEAM_REP / SUPER_ADMIN	/api/matches/{id} /api/matches/{id}/lineups/{teamId} /api/matches/{id}/protocol	Открытие матча, редактирование заявки, сохранение протокола	Официальная карточка матча, lineups и protocol workflow.
Players.vue	Публично	/api/players /api/players/{id} /api/players/{id}/history	Открытие страницы, поиск, пагинация, открытие модалки	Каталог игроков и подробная player card.
Admin.vue	SUPER_ADMIN	Seasons API, Teams API, Tours API, Players API, Admin Access API	Переключение вкладок, submit форм, кнопки публикации/назначения/удаления	Полное административное управление турниром.
TeamRepDashboard.vue	TEAM_REP	/api/team-rep/* /api/teams/{teamId}/players/{playerId}	Открытие кабинета, выбор сезона, действия по составу	Работа с командой и заявкой на сезон.
ApiExplorer.vue	Фактически публично	HTTP нет	Открытие страницы	Справочник endpoint-ов по статическому каталогу.
ApiExplorerTest.vue	Зависит от endpoint-a	Любой endpoint из каталога	Кнопка «Тест»	Ручной прогон реального запроса к backend.
Teams.vue	Публично	HTTP нет	Открытие страницы	Legacy-витрина на локальном store.
CreateMatch.vue	Auth required	HTTP нет	Submit формы	Legacy/demo flow создания матча, не production контур.

13. Замечания для QA

Различать production и legacy

Текущая боевая логика турниров, игроков, матча, админки и team representative идет через backend API. Страницы /teams , /create и Home.vue не должны использоваться как эталон текущей предметной модели.

Бан пользователя сейчас не серверный

Вкладка «Бан» в админке работает на `localStorage`. Она не блокирует реальные backend-вызовы и не является полноценной системой безопасности.

API Explorer route доступнее, чем menu link

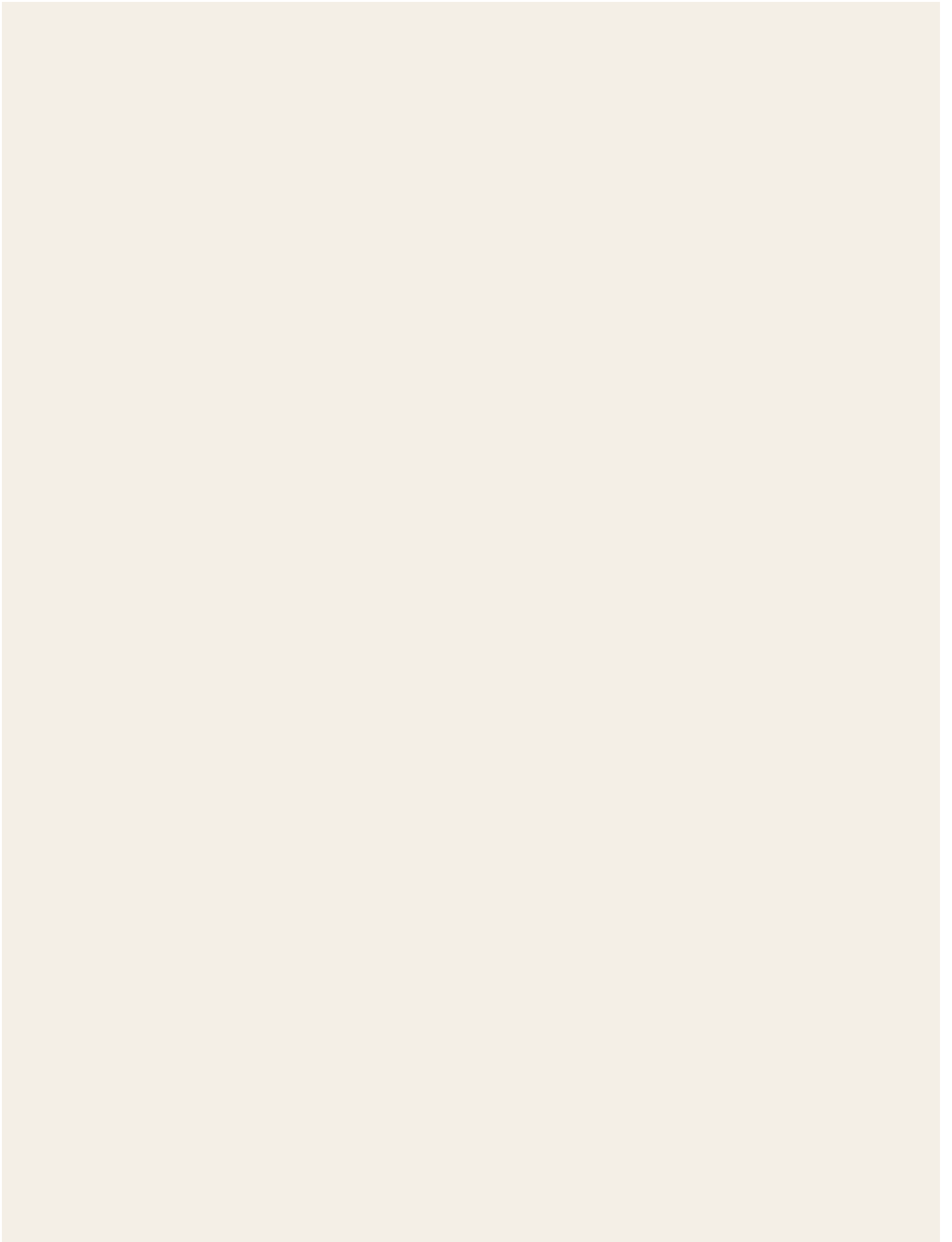
В навигации ссылка на API Explorer скрыта для не-админа, но сам route сейчас не закрыт router guard-ом.

Игроки публично доступны в текущем runtime

Это наблюдаемое поведение стоит отдельно подтвердить security-правилами backend, потому что каталог API местами описывает эти endpoint-ы строже.

Рекомендуемый приоритет тестирования

1) главная страница и season overview, 2) карточка матча и протокол, 3) player catalog и modal, 4) admin season/team/tour flows, 5) кабинет TEAM_REP, 6) API Explorer как вспомогательный инструмент проверки контрактов.



Документ подготовлен как статический HTML-файл внутри `public`, чтобы его можно было открыть локально через dev/preview сервер по прямому URL без дополнительных роутов и без сборки отдельного документационного сайта.